

LESSON 12

Next.js Routing & Page Setup

WEEK 03

Introduction to Next.js

❖ What is Next.js?

- A React framework for building web applications with server-side rendering, static site generation, and API routes. Supports hybrid rendering: SSR, SSG, CSR.

❖ Key Benefits

- Automatic code splitting, optimized performance, built-in CSS support, and easy deployment. Used by companies like Vercel, Netflix for scalable apps.

❖ Version 15 Highlights

- Stable integration with React 19, improved caching, and Turbopack for faster builds.

Evolution of Routing in Next.js

❖ Pages Router (Legacy)

- File-based routing in /pages directory, simple but limited in nested layouts and data fetching. Example: /pages/about.js maps to /about.

❖ App Router (New Standard)

- Introduced in Next.js 13, enhanced in 15; uses /app directory for more flexible, React Server Components-focused routing. Supports streaming, suspense, and better error handling.

❖ Why Migrate?

- Better performance, colocation of data fetching, and shared layouts without re-renders.

What is App Router?

❖ Core Concept

- File-system based router using folders in /app to define routes and UI. Built on React Server Components, Suspense, and Server Actions.

❖ Features Overview

- Nested routes, dynamic segments, layouts, and integrated data fetching. Defaults to server-side rendering for better SEO and performance.

❖ New in Next.js 15

- Opinionated caching defaults for routes, reducing complexity in data management.

Setting Up App Router

❖ Project Setup

- Create a new Next.js app with `npx create-next-app@latest` and enable App Router. Opt for TypeScript and Tailwind for better development.

❖ Directory Structure

- `/app` as root; includes files like `page.tsx`, `layout.tsx`. Ignore `_` prefixed folders/files for private use.

❖ Running Locally

- Use `npm run dev` to see routes in action.

Defining Basic Routes

❖ Folder-Based Routing

- Folders map to URL paths; page.tsx defines the UI for that route.
Example: /app/dashboard/page.tsx -> /dashboard.

❖ Static Routes

- Simple, no params; ideal for fixed pages like home or about. Exports a default React component.

❖ Client vs Server

- Defaults to Server Components; use 'use client' for interactivity.

Nested Routes

❖ Nesting Folders

- Create sub-routes by nesting folders, e.g., /app/blog/posts/page.tsx -> /blog/posts. Allows hierarchical URL structures.

❖ Benefits

- Organizes code logically, shares layouts between nested pages. Navigation preserves state in parent layouts.

❖ Implementation

- Each nested folder can have its own page.tsx and layout.tsx.

Dynamic Routes

❖ Dynamic Segments

- Use [param] for folders, e.g., /app/products/[id]/page.tsx -> /products/123. Access via params prop: { params: { id: '123' } }.

❖ Usage

- For data-driven pages like product details or user profiles. Supports async params in Server Components.

❖ New in 15

- Better handling with React 19 for dynamic data fetching.

Catch-All Routes

❖ Catch-All Segments

- Use [...param] for catching multiple segments, e.g.,
/app/docs/[...slug]/page.tsx -> /docs/a/b/c. Params: { slug: ['a', 'b', 'c'] }.

❖ Optional Catch-All

- [[...param]] includes the root path. Useful for optional params or 404 handling.

❖ Benefits

- Flexible for variable URL depths like documentation sites.

Layouts in App Router

❖ Layout Files

- layout.tsx defines shared UI wrapping children (pages or nested layouts). Preserves state on navigation, no re-renders.

❖ Root Layout

- Mandatory, includes `<html>` and `<body>`. Can add metadata, fonts, etc.

❖ Nested Layouts

- Applied hierarchically; parent wraps child.

Shared and Dynamic Layouts

❖ Shared Layouts

- One layout applies to multiple routes in the same folder level. Reduces code duplication.

❖ Dynamic Layouts

- Access params for conditional rendering. Example: User-specific sidebar based on param.

❖ Composition

- Layouts compose via children prop.

Templates vs Layouts

❖ Templates

- Similar to layouts but re-render on navigation, resetting state. Use `template.tsx` for cases needing fresh state.

❖ Differences

- Layouts preserve state; templates do not. Ideal for modals or animations that reset.

❖ Usage

- Export React component with children.

Loading UI

❖ Loading Files

- loading.tsx shows instant loading state during data fetch. Uses React Suspense for streaming.

❖ Benefits

- Improves UX by showing skeletons before content loads. Applies to route segment and children.

❖ Implementation

- Export component; wraps page in Suspense.

Error Handling

❖ Error Files

- error.tsx catches errors in route segments. Provides fallback UI and reset function.

❖ Global Error

- global-error.tsx for root-level errors. New in 15: Redesigned error UI with better stack traces.

❖ Usage

- 'use client' for interactivity; access error and reset props.

Not Found Page

❖ Not-Found Files

- not-found.tsx for custom 404 pages. Triggered by notFound() function or unmatched routes.

❖ Benefits

- Custom UX for missing content. Can be nested for segment-specific 404s.

❖ Implementation

- Export component; use notFound() in code.

Route Groups

❖ Grouping Routes

- Use (group) folders to organize without affecting URL.
- Example: /app/(auth)/login/page.tsx -> /login (no '/auth' in URL).

❖ Benefits

- Logical separation, e.g., auth vs marketing routes.
- Can have separate layouts per group.

❖ Example

- app/(marketing)/page.tsx
- app/(app)/dashboard/page.tsx.

Parallel Routes

❖ Parallel Rendering

- Use @folder for slots rendering independently.
- Example: /app/@analytics/page.tsx for side-by-side content.

❖ Usage

- Dashboards with multiple views; conditional via children prop.
- Supports unmatched routes with default.tsx.

❖ Navigation

- Independent loading and errors.

Intercepting Routes

❖ Intercepting

- Use (.)segment to intercept routes for modals/overlays.
- Example: /app/(..)photos/page.tsx intercepts /photos in modal.

❖ Soft vs Hard Navigation

- Soft keeps context; hard is full page. Useful for modals without losing page state.

❖ Patterns

- (.) for same level, (..) for parent, etc.

Colocation in App Router

❖ Colocation

- Place components, tests, styles in /app folders without creating routes.
- Only special files (page.tsx, etc.) define routes.

❖ Benefits

- Keeps related files together, e.g., utils next to page.
- Private folders with _ prefix for non-public files.

❖ Example

- /app/blog/components/PostCard.tsx (no route).

Data Fetching in Routing

❖ Integrated Fetching

- Fetch data in Server Components/pages.
- Use native fetch with extended options.

❖ Strategies

- SSR (default), SSG (generateStaticParams), ISR.
- Async components for awaiting data.

❖ Search Params

- Access via searchParams prop for filtering.

Streaming in App Router

❖ Streaming

- Render UI incrementally with Suspense boundaries. Fallback to loading while data streams in.

❖ Benefits

- Faster TTFB, better for slow networks. Loading.tsx uses Suspense automatically.

❖ New in 15

- Enhanced with React 19 for better partial hydration.

Suspense Boundaries

❖ Suspense Usage

- Wrap async parts for loading states. Multiple boundaries for granular control.

❖ Integration with Routes:

- In layouts or pages for per-segment loading.
- Error handling per boundary.

❖ Example

- Streaming user data while rendering static parts.

Caching in Next.js 15

❖ Caching Defaults

- Opinionated defaults in App Router: cache fetches by default.
- Router Cache: Client-side for navigation (changed in 15: no reuse for page segments).

❖ Types

- Data Cache (persistent), Full Route Cache (build time). Opt-out with no-store.

❖ New in 15

- Simpler defaults, better integration with dynamic routes.

Revalidation

❖ Revalidate Data

- Use revalidate for ISR, or revalidatePath/revalidateTag. Time-based or on-demand.

❖ Usage in Routes

- For dynamic routes with stale-while-revalidate.
- Example: News pages revalidate every 60s.

❖ Implementation

- Export const revalidate = 60;

Route Handlers

❖ API in App Router

- Use route.ts for API endpoints.
- Supports HTTP methods: GET, POST, etc.

❖ Usage

- /app/api/route.ts -> /api.
- Dynamic**: With params like pages.

❖ Benefits

- Server-only, no bundle size increase.

Middleware

❖ Middleware

- middleware.ts at root for request interception. Rewrite, redirect, headers.

❖ Routing Integration

- Protect routes, A/B testing.
- Matcher for specific paths.

❖ Example

- Auth check before rendering.

Internationalized Routing

❖ **i18n Routing**

- Config in next.config.js for locales. Subpath or domain routing.

❖ **App Router Support**

- Dynamic locales in params. Use generateStaticParams.

❖ **Example**

- `/app/[locale]/page.tsx`.

Common Pitfalls

❖ Pitfalls

- Forgetting root layout, misuse client directives. Caching traps: Forgetting revalidate for dynamic data.

❖ Debugging

- Use new error UI in v15.
- Avoid nested errors without boundaries.

❖ Migration Issues

- Incremental adoption from Pages.